

OSSP

Skill Week-1

Roll No-2520090085

Name -: Vakkalanka Sai Bhaskara Sujit

Task 1: To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

Source Code-:

```
GNU nano 8.7.1 Task1.c
#include <stdio.h>
#include <string.h>

#define MAX_LEN 200
#define MAX_QUOTES 20

int main() {
    char input[MAX_LEN];
    char quoted_strings[MAX_QUOTES][MAX_LEN];
    int quote_count = 0;

    printf("Enter a command with single quotes:\n");
    printf("mysHELL> ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    int i = 0;
    while (input[i] != '\0') {
        if (input[i] == '\\') {
            i++;
            int j = 0;
            while (input[i] != '\'' && input[i] != '\0') {
                if (j < MAX_LEN - 1) {
                    quoted_strings[quote_count][j++] = input[i];
                }
            }
            quote_count++;
        }
        i++;
    }
}
```

```
GNU nano 8.7.1 Task1.c
#include <stdio.h>
#include <string.h>

#define MAX_LEN 200
#define MAX_QUOTES 20

int main() {
    char input[MAX_LEN];
    char quoted_strings[MAX_QUOTES][MAX_LEN];
    int quote_count = 0;

    printf("Enter a command with single quotes:\n");
    printf("mysHELL> ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    int i = 0;
    while (input[i] != '\0') {
        if (input[i] == '\\') {
            i++;
            int j = 0;
            while (input[i] != '\'' && input[i] != '\0') {
                if (j < MAX_LEN - 1) {
                    quoted_strings[quote_count][j++] = input[i];
                }
            }
            quote_count++;
        }
        i++;
    }
}
```

```
GNU nano 8.7.1 Task1.c
    if (j < MAX_LEN - 1) {
        quoted_strings[quote_count][j++] = input[i];
    }
    i++;
    quoted_strings[quote_count][j] = '\0';

    if (input[i] == '\') {
        quote_count++;
        i++;
    } else {
        printf("Error: Unclosed single quote\n");
    }
    i++;
}

printf("\nQuoted strings found:\n");
if (quote_count == 0) {
    printf("No single-quoted strings found\n");
} else {
    for (int k = 0; k < quote_count; k++) {
        printf("Quoted String %d: [%s]\n", k + 1, quoted_strings[k]);
    }
}

printf("\nParsing validation:\n");
if (quote_count > 0) {
    printf("Single-quote parsing successful\n");
} else {
    printf("No quoted content to validate\n");
}

return 0;
```

Output:-

```
Task1.c:58:5: error: expected declaration or statement at end of input
 58 |     return 0;
    |     ^~~~~~
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ nano Task1.c
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ gcc Task1.c -o task
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ ./task
Enter a command with single quotes:
myshell> echo 'Hello $USER'

Quoted strings found:
Quoted String 1: [Hello $USER]

Parsing validation:
Single-quote parsing successful
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$
```

Task 2: Double Quote Parsing & Variable Expansion

Source Code:-

```
nocturnal@nocturnal: /mnt/c, × + v
GNU nano 8.7.1 Taskk2.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 100

void expand_variable(char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_INPUT - 1) {
        if (input[i] == '$') {
            i++;
            char variable[50];
            int k = 0;

            while (isalnum((unsigned char)input[i]) || input[i] == '_') {
                if (k < 49) {
                    variable[k++] = input[i];
                }
                i++;
            }
            variable[k] = '\0';
        }
    }
}
```

```
nocturnal@nocturnal: /mnt/c, × + v
GNU nano 8.7.1 Taskk2.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 100

void expand_variable(char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_INPUT - 1) {
        if (input[i] == '$') {
            i++;
            char variable[50];
            int k = 0;

            while (isalnum((unsigned char)input[i]) || input[i] == '_') {
                if (k < 49) {
                    variable[k++] = input[i];
                }
                i++;
            }
            variable[k] = '\0';
        }
    }
}
```

```

fgets(input, sizeof(input), stdin);
input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0) {
    printf("Empty command\n");
    return 0;
}

while (input[i] != '\0' && token_count < MAX_TOKENS) {
    while (isspace((unsigned char)input[i])) {
        i++;
    }

    if (input[i] == '\0') break;

    int j = 0;
    if (input[i] == '"') {
        i++;
        while (input[i] != '\0' && input[i] != '"') {
            if (j < MAX_TOKEN_LENGTH - 1) {
                tokens[token_count][j++] = input[i];
            }
            i++;
        }
        if (input[i] == '"') {
            i++;
        }
    }
}

```

Help Write Out Where Is Cut Execute Location Undo Set

```

fgets(input, sizeof(input), stdin);
input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0) {
    printf("Empty command\n");
    return 0;
}

while (input[i] != '\0' && token_count < MAX_TOKENS) {
    while (isspace((unsigned char)input[i])) {
        i++;
    }

    if (input[i] == '\0') break;

    int j = 0;
    if (input[i] == '"') {
        i++;
        while (input[i] != '\0' && input[i] != '"') {
            if (j < MAX_TOKEN_LENGTH - 1) {
                tokens[token_count][j++] = input[i];
            }
            i++;
        }
        if (input[i] == '"') {
            i++;
        }
    }
}

```

Help Write Out Where Is Cut Execute Location Undo Set

```

fgets(input, sizeof(input), stdin);
input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0) {
    printf("Empty command\n");
    return 0;
}

while (input[i] != '\0' && token_count < MAX_TOKENS) {
    while (isspace((unsigned char)input[i])) {
        i++;
    }

    if (input[i] == '\0') break;

    int j = 0;
    if (input[i] == '"') {
        i++;
        while (input[i] != '\0' && input[i] != '"') {
            if (j < MAX_TOKEN_LENGTH - 1) {
                tokens[token_count][j++] = input[i];
            }
            i++;
        }
        if (input[i] == '"') {
            i++;
        }
    }
}

```

Help Write Out Where Is Cut Execute Location Undo Set

Output-:

```
nocturnal@nocturnal:/mnt/c/Users/vsbsu$ gcc task2.c -o task1
nocturnal@nocturnal:/mnt/c/Users/vsbsu$ ./task1
Enter a command:
myshell> ps

Tokens before variable expansion:
Token 1: [ps]

Variable expansion:
Token 1: [ps]

Parsing validation:
Double-quote parsing successful
Spaces inside quotes were preserved
Variable expansion was processed
nocturnal@nocturnal:/mnt/c/Users/vsbsu$ |
```

Escape Sequence Parsing

```
GNU nano 8.7.1 task3.c
#include <stdio.h>
#include <string.h>

#define MAX_LEN 200

int main() {
    char input[MAX_LEN];
    char output[MAX_LEN];
    int i = 0;
    int j = 0;
    int escaped = 0;

    printf("Enter a command with escaped spaces or special characters:\n");
    printf("myshell> ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty input\n");
        return 0;
    }

    while (input[i] != '\0' && j < MAX_LEN - 1) {
        if (escaped) {
            output[j++] = input[i];
        }
    }
}
```

```
GNU nano 8.7.1 Taskk3.c
printf("Empty input\n");
return 0;
}

while (input[i] != '\0' && j < MAX_LEN - 1) {
    if (escaped) {
        output[j++] = input[i];
        escaped = 0;
    } else if (input[i] == '\\') {
        escaped = 1;
    } else {
        output[j++] = input[i];
    }
    i++;
}

if (escaped) {
    printf("\nWarning: Escape character at the end of input\n");
}

output[j] = '\0';

printf("\nOriginal Input:\n%s\n", input);
printf("\nProcessed Input:\n%s\n", output);

printf("\nParser validation:\n");
```

```
GNU nano 8.7.1 Taskk3.c
printf("Empty input\n");
return 0;
}

while (input[i] != '\0' && j < MAX_LEN - 1) {
    if (escaped) {
        output[j++] = input[i];
        escaped = 0;
    } else if (input[i] == '\\') {
        escaped = 1;
    } else {
        output[j++] = input[i];
    }
    i++;
}

if (escaped) {
    printf("\nWarning: Escape character at the end of input\n");
}

output[j] = '\0';

printf("\nOriginal Input:\n%s\n", input);
printf("\nProcessed Input:\n%s\n", output);

printf("\nParser validation:\n");
```

Output:-

```

nocturnal@nocturnal:/mnt/c/Users/vsbsu$ nano Taskk3.c
nocturnal@nocturnal:/mnt/c/Users/vsbsu$ gcc Taskk3.c -o task1
nocturnal@nocturnal:/mnt/c/Users/vsbsu$ ./task1
Enter a command with escaped spaces or special characters:
myshell> hello\ world

Original Input:
hello\ world

Processed Input:
hello world

Parser validation:
Escape sequences processed successfully
Escaped characters were preserved
nocturnal@nocturnal:/mnt/c/Users/vsbsu$ |

```

Task : Process Execution (fork, exec, waitpid)

Source Code-:

```

GNU nano 8.7.1 Taskk4.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Executing: ls -l\n");
        execlp("ls", "ls", "-l", NULL);
    }
}

```

```
GNU nano 8.7.1 Taskk4.c *
if (pid == 0) {
    // Child process
    printf("\nChild Process\n");
    printf("Child PID: %d\n", getpid());
    printf("Parent PID: %d\n", getppid());

    printf("Executing: ls -l\n");
    execlp("ls", "ls", "-l", NULL);

    // If execlp fails
    perror("exec failed");
    exit(EXIT_FAILURE);
} else {
    // Parent process
    printf("\nParent Process\n");
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);
    printf("Parent waiting for child...\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("\nChild exited with status: %d\n", WEXITSTATUS(status));
    } else {
        printf("\nChild terminated abnormally\n");
    }
}
```

```
GNU nano 8.7.1 Taskk4.c *
if (pid == 0) {
    // Child process
    printf("\nChild Process\n");
    printf("Child PID: %d\n", getpid());
    printf("Parent PID: %d\n", getppid());

    printf("Executing: ls -l\n");
    execlp("ls", "ls", "-l", NULL);

    // If execlp fails
    perror("exec failed");
    exit(EXIT_FAILURE);
} else {
    // Parent process
    printf("\nParent Process\n");
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);
    printf("Parent waiting for child...\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("\nChild exited with status: %d\n", WEXITSTATUS(status));
    } else {
        printf("\nChild terminated abnormally\n");
    }
}
```

Output:-

```
cturnal@nocturnal:/mnt/c/Users/vsbsu$ gcc Taskk4.c -o task1
cturnal@nocturnal:/mnt/c/Users/vsbsu$ nano Taskk4.c
cturnal@nocturnal:/mnt/c/Users/vsbsu$ ./Taskk4.c
Taskk4.c: line 7: syntax error near unexpected token `('
Taskk4.c: line 7: `int main() {'
cturnal@nocturnal:/mnt/c/Users/vsbsu$ ./Task1

Parent Process
Parent PID: 2008
Child PID: 2009
Parent waiting for child...

Child Process
Child PID: 2009
Parent PID: 2008
Executing: ls -l
total 18600
wxrwxrwx 1 nocturnal nocturnal 4096 Jul 29 19:22 AppData
wxrwxrwx 1 nocturnal nocturnal 34 Jul 29 19:22 'Application Data' -> /mnt/c/Users/vsbsu/AppData/Roaming
wxrwxrwx 1 nocturnal nocturnal 4096 Jul 29 19:24 Contacts
wxrwxrwx 1 nocturnal nocturnal 62 Jul 29 19:22 Cookies -> /mnt/c/Users/vsbsu/AppData/Local/Microsoft/Windows/IN
Cookies
wxrwxrwx 1 nocturnal nocturnal 4096 Aug 8 13:27 CrossDevice
wxrwxrwx 1 nocturnal nocturnal 4096 Aug 14 16:34 Desktop
wxrwxrwx 1 nocturnal nocturnal 4096 Aug 8 17:51 Documents
wxrwxrwx 1 nocturnal nocturnal 4096 Aug 14 17:13 Downloads
wxrwxrwx 1 nocturnal nocturnal 4096 Jul 29 19:24 Favorites
wxrwxrwx 1 nocturnal nocturnal 4096 Jul 29 19:24 Links
wxrwxrwx 1 nocturnal nocturnal 32 Jul 29 19:22 'Local Settings' -> /mnt/c/Users/vsbsu/AppData/Local
```

